

Learning Cluster-Scheduling Policies that Improve Tail Latency

Title page

Title: Learning Cluster-Scheduling Policies that Improve Tail Latency

Team members (Cornell NetIDs):

- Frank Dai — sd924
- Johnson Wang — jw2693
- Jerry Ji — rj378

AI keywords: Reinforcement learning, Proximal Policy Optimization (PPO), action masking, deep learning, sequential decision making, neural attention.

Application setting: Online cluster scheduling — deciding, for each arriving compute job, which machine in a multi-machine cluster should run it so that user-visible waiting-time metrics (especially the 95th- and 99th- percentile tails) are minimized. We build a discrete-event cluster simulator to model this and compare a learned policy against three classical heuristics across six controlled workload regimes.

Other contributors: None outside the listed team. (No external proofreaders or user-study participants were used; the evaluation is fully automated and runs against simulated workloads.)

Part 1: Project description

1.1 What we had planned to do

In the original proposal we set out to study reinforcement learning for an **online cluster-scheduling problem**. The motivation was straightforward: classical scheduling heuristics like First-Fit, Best-Fit, and Shortest-Job- First (SJF) make decisions based on a single rule of thumb each, but a production cluster’s user-visible quality of service depends on getting many decisions right at once — picking the *right* machine, packing well, not starving long jobs, and reacting to load spikes. We hypothesized that a deep RL policy, trained against a faithful simulator, could learn to balance these concerns better than any single heuristic.

Concretely, the proposal targeted three deliverables:

1. A **discrete-event cluster simulator** that models machines with finite CPU and memory, jobs with stochastic arrivals and demands, and a waiting queue with completion events that

drive time forward.

2. A library of **heuristic baselines** (First-Fit, Best-Fit, SJF) that plug into the simulator through a common interface.
3. An **RL policy** trained via PPO that takes the cluster’s state as input and outputs a placement action, evaluated on the same workloads as the heuristics so that comparisons are fair.

The original evaluation plan was to run all schedulers on a single canonical workload, report mean and p95 waiting time and overall cluster utilization, and declare success if the RL policy beat First-Fit on at least one of those metrics by a statistically meaningful margin.

1.2 What the project wound up being

The shape of the project is broadly consistent with the proposal — a simulator, heuristic baselines, and a learned policy — but several elements grew, one was scoped down, and one new line of work was added and then documented as a negative result.

Grew: the evaluation breadth. A single workload turned out to be a poor way to differentiate schedulers. At low load, every scheduler is optimal (no waiting). At high load, schedulers’ relative strengths and weaknesses emerge but in different ways depending on the workload’s *shape*: bursty arrivals stress one set of decisions, large jobs stress another, small clusters yet another. We expanded to six controlled regimes — `light_poisson`, `heavy_poisson`, `bursty`, `large_jobs`, `small_cluster`, `wide_cluster` — and reported results across all of them with five held-out evaluation seeds and 95% confidence intervals. This made the comparison statistically meaningful and exposed regimes where SJF, despite winning on mean time, was catastrophically bad on the tail.

Grew: the RL recipe. The simplest formulation — PPO on a flat observation, with reward `-waiting_time` — collapsed in training because PPO without action masking spends probability on infeasible machines and gets stuck in local optima. We adopted **MaskablePPO** from `sb3-contrib`, which restricts the policy distribution to feasible actions at every step. We further added (a) a **richer featurizer** that exposes per-machine post-placement headroom, top-k queued-job features, and backlog totals; (b) **reward shaping** with a per-step backlog penalty and a one-time terminal completion bonus; (c) **VecNormalize** on returns to stabilize the value-function target scale; (d) a **linear learning-rate schedule** that decays to zero over training. Each of these was added in response to observed training failures or weak evaluation results, not pre-planned.

Scoped down: a single cross-cluster policy. The proposal envisioned a single RL policy that could be evaluated on different cluster sizes. This is hard because PPO’s action space (`Discrete(num_machines)`) is fixed at policy-construction time. We built three policies instead — one for the 4-machine, 10-machine, and 20-machine regimes — sharing hyperparameters but not weights. The evaluation harness routes the right policy to each regime automatically based on a metadata sidecar that travels with each saved model.

Added then documented as a negative result: a set-attention policy for cross-cluster generalization. Late in the project we attempted to recover the proposal’s “one policy for any cluster size” goal by designing a custom MaskablePPO policy with (a) a permutation-invariant attention-based feature extractor that treats per-machine slots as an unordered set, and (b) a

pointer-network-style actor head whose scoring weights are shared across machines. Unit tests pass — the extractor is permutation-invariant, padded slots do not affect active embeddings, and a model trained at `num_machines=3` can predict on `num_machines=5` through the same checkpoint. Full PPO training, however, reliably collapses to a near-deterministic policy in the first update across many hyperparameter combinations. We document this in Section 1.3 below as honest failure rather than abandoning it silently.

1.3 Key aspects of AI at the core of the project

This section walks through the AI ideas that drove the project, in the order they appear when training and evaluating a policy.

1.3.1 Markov decision process formulation

Cluster scheduling fits naturally into the MDP framework. We define:

- **State s :** a snapshot of all machines’ CPU and memory utilization, the current job awaiting placement, and the queue of jobs that have arrived but cannot yet fit anywhere.
- **Action a :** an integer in $\{0, \dots, M-1\}$ selecting a machine. The environment supplies an action mask: machine i is masked off whenever it cannot fit the current job. Under masking the policy’s effective action space is the set of fittable machines, which can be empty.
- **Transition:** the chosen machine begins running the job immediately; the simulator advances to the next discrete event, which is the earlier of the next arrival or the next completion. Jobs that arrive while no machine can hold them join the wait queue and are reconsidered after each completion.
- **Reward:** the dense per-step reward is $-\text{waiting_time} - \lambda_b \cdot |\text{queue}|$, where waiting_time is the placed job’s queue delay and $\lambda_b = 0.1$ is a backlog-penalty coefficient. A one-time bonus of $+1.0$ is added on the final step. The negative-waiting term incentivizes fast placement; the backlog term encourages draining the queue rather than locally optimal greedy choices that let the queue grow.

This formulation reduces cluster scheduling to a familiar discrete-action episodic RL problem and admits standard policy-gradient methods directly.

1.3.2 Action masking under MaskablePPO

PPO is the standard policy-gradient method for discrete action spaces, but the vanilla algorithm spends probability mass on infeasible actions: the categorical distribution covers every machine, including those that cannot hold the current job. In the worst case the policy samples an illegal action, the environment rejects it, and a placement opportunity is wasted. In the best case, illegal actions get a prescribed penalty and the policy learns to avoid them — but at the cost of much longer training.

MaskablePPO [Huang & Ontañón 2022] takes a state-dependent boolean mask and zeros out the corresponding logits before the softmax, so the categorical distribution is restricted exactly to feasible actions. The PPO update then has zero gradient through masked positions, eliminating

wasted optimization on illegal placements. Adopting MaskablePPO turned a brittle, non-converging training run into a stable one and is the single largest contributor to the project’s success.

1.3.3 Featurization

The choice of state representation is a first-order design decision in deep RL. A naïve flat encoding — concatenating raw machine and job features — worked but plateaued well below the heuristics. We designed a “rich” featurizer that exposes:

- Per-machine **utilization** (CPU and memory ratios in $[0, 1]$).
- The **current job**’s normalized CPU, memory, and duration.
- A **top-k queue peek**: the next four queued jobs’ (CPU, memory, duration), zero-padded when fewer than four are present.
- **Backlog totals**: total CPU and total memory demand summed over the wait queue, normalized by total cluster capacity.
- **Per-machine post-placement headroom**: how much CPU and memory each machine would have *after* accepting the current job. This feature is the most physically meaningful one — it directly tells the policy “if you place here, you’ll still be able to fit jobs in the future.”

The full observation is clipped to $[0, 1]$ so all features lie on a common scale, and a fixed `DURATION_REF = 100.0` constant is used to normalize durations so observations are comparable across workloads with different duration distributions. The featurizer is wired into both training (in the Gymnasium environment) and evaluation (in the `RLScheduler` adapter) so observations are byte-for-byte identical in both phases.

1.3.4 Reward shaping and stability

The default reward (`-waiting_time`) gives a learning signal but is sparse: a placement at time t is rewarded based only on its own queue delay, not on whether queueing is building up cluster-wide. We added two shaping terms:

- A **backlog penalty** $-\lambda_b \cdot |\text{queue}|$ per step encourages global drainage. The policy now sees a small negative signal whenever the queue is growing, even before any individual job has waited long.
- A **completion bonus** $+ b_c$ on the terminal step explicitly rewards finishing the workload. This counters episodes where the policy’s value function attaches negative value to terminal states.

Reward magnitudes can swing wildly across regimes (a heavy-load run generates returns $\sim 10\times$ those of a light run). We enabled `VecNormalize` on returns, which divides each step’s reward by a running standard deviation. This keeps the value function’s optimization target on a stable scale and prevents early gradient explosions that we observed in unstable training runs. The running statistics are saved alongside each model so any later resume can pick up the calibrated scale rather than restart it.

A **linear learning-rate schedule** decaying from 2.5×10^{-4} to 0 over training trades exploration early for exploitation late. We also clip the value-function loss (`clip_range_vf = 0.2`), which

empirically prevents the critic from overshooting on large advantage estimates early in training.

1.3.5 Policy network

The policy is a two-layer MLP with 256-unit GeLU hidden layers in each branch (actor and critic). The architecture is small by deep-RL standards but sufficient given the modest observation dimensionality (~50 features for the 10-machine **RichFeaturizer**). Training takes ~14 minutes per cluster size on a 12-core M4 Pro with 10 parallel SubprocVecEnv workers, which made hyperparameter iteration fast.

1.3.6 Heuristic baselines

We implemented three classical online schedulers as baselines:

- **First-Fit (FF)**: scan machines in id order; place on the first that fits. No global view; simple and fast.
- **Best-Fit (BF)**: scan all machines; place on the one with the smallest total free CPU + memory after placement. Reduces fragmentation; one extra pass per decision.
- **Shortest-Job-First (SJF)**: sort the queue by job duration; place the shortest fittable job on the first fittable machine. SJF is the **oracle greedy** for minimizing mean waiting time when durations are known in advance, which we provide. It is expected to perform poorly on tail metrics: long jobs systematically defer to short ones and accumulate large waiting times.

These three baselines stress the comparison from different angles: FF gives a “naïve” floor, BF tests whether smart packing matters at all, and SJF tests whether oracle duration knowledge is sufficient or whether the policy must reason holistically about the queue.

1.3.7 Set-attention extension (negative result)

To attempt a single cross-cluster policy we designed a permutation- invariant alternative architecture with three components:

1. A **set-aware featurizer** (**SetFeaturizer**) that pads the observation to a fixed `MAX_MACHINES = 32` and adds an `is_active` bit per slot.
2. A **multi-head self-attention extractor** that embeds each machine with a shared MLP, injects the current job as context, and runs two layers of attention with the `is_active` mask as the key-padding mask. Padded slots do not contribute, by construction.
3. A **pointer-network actor head**: a single shared linear layer applied to each machine’s embedding produces one scalar logit per machine. A masked mean-pool of active machine embeddings drives the critic head.

Together these three pieces are permutation-invariant and naturally generalize to any cluster size up to `MAX_MACHINES`, since every component operates on a per-machine basis with weights shared across slots.

The unit-test suite exercises this architecture: the extractor’s output for active slots is invariant to which padded slots are present, and a model trained on a 3-machine cluster successfully predicts on a 5-machine cluster through the same `MaskablePP0.load` path. **Full training, however, collapses.**

Across many configurations of reward scale, learning rate, entropy coefficient, and `VecNormalize` on/off, the policy reliably reports an `entropy_loss` of about -0.14 (entropy near 0.14 nats — near-deterministic) after a single PPO update, with `approx_kl` on the order of 10^{-5} to 10^{-7} ; gradients then vanish and training stalls. We attempted small-gain orthogonal initialization on the custom heads (the standard fix for actor collapse) and per-step reward rescaling without success. The most likely culprits are either an interaction between SB3’s shared features-extractor and our custom heads’ gradient flow, or a subtle gradient-zero condition where the action mask is very restrictive. Diagnosing the exact cause was not possible within the project window. The architecture is left in the repository (`cluster_scheduler/policies.py`, `tests/test_policies.py`) with passing unit tests as future work.

Part 2: Evaluation

This section is the substantive scientific evaluation of the project. It is organized as the syllabus requests: (1) what questions we asked, (2) how we went about answering them, (3) the specific quantitative answers we got. Each subsection ends with a short “Answer” paragraph stating what we conclude.

2.1 Research questions

Our evaluation targets four questions that, taken together, decide whether the project succeeded:

- **Q1 — Tail-latency win versus oblivious heuristics.** Does the learned policy beat First-Fit and Best-Fit on the 95th- and 99th-percentile waiting time across multiple workload regimes?
- **Q2 — Mean-versus-tail trade-off versus the duration oracle.** Shortest-Job-First is given exact job durations and is the optimal greedy policy for mean waiting time. Does it dominate the learned policy across all metrics, or is there a trade where RL gives up some mean for substantial tail-latency improvement?
- **Q3 — Source of gains.** Are the learned policy’s wins attributable to better admission control (fitting more jobs in less time) or to better packing/ordering of the same set of jobs? In other words, do schedulers achieve different *cluster utilizations*, or do they achieve the same utilization with different waiting-time profiles?
- **Q4 — Recipe robustness across cluster sizes.** Does the same training recipe — featurizer, reward shape, hyperparameters — produce a working policy at multiple cluster sizes (4, 10, 20 machines), or does each setting require bespoke tuning?

2.2 Methodology

2.2.1 Workload regimes

We evaluate on six regimes, each defined by a cluster shape and a workload generator. Arrival rates are calibrated so each regime operates near 60–80% utilization (well into the regime where

queueing becomes significant) under the heuristic baselines, ensuring there is meaningful work for any scheduler to do:

- **light_poisson**: 10 machines, 200 jobs, arrival rate 5.0.
- **heavy_poisson**: 10 machines, 200 jobs, arrival rate 8.0.
- **bursty**: 10 machines, 200 jobs, base rate 4.0, bursts to 12.0 with probability 0.15 and length 8.
- **large_jobs**: 10 machines, 150 jobs, arrival rate 3.0, with larger CPU and memory demands per job to stress packing.
- **small_cluster**: 4 machines, 150 jobs, arrival rate 2.0.
- **wide_cluster**: 20 machines, 400 jobs, arrival rate 6.0. This regime turned out to be under-saturated at this rate (every scheduler hits 0 wait); we discuss this in Section 2.3 as a known limitation rather than a meaningful comparison point.

2.2.2 Trained policies

Three policies, trained with the same recipe (**RichFeaturizer**, dense reward with backlog penalty 0.1 and completion bonus 1.0, **VecNormalize**, linear LR schedule from 2.5×10^{-4} to 0, two-layer 256-unit GeLU MLP, 10 SubprocVecEnv workers, 8 million environment steps), differing only in cluster size:

- **ppo_m4_v4**: 4-machine cluster, training arrival rate 2.0.
- **ppo_m10_v4**: 10-machine cluster, training arrival rate 8.0.
- **ppo_m20_v4**: 20-machine cluster, training arrival rate 6.0.

Training seed is 0; evaluation seeds are 1000–1004, disjoint from training.

2.2.3 Statistical methodology

Each workload trace is generated once per (**regime**, **seed**) cell and shared across **all four** schedulers under test, eliminating workload-noise as a confound when comparing schedulers. We run five independent seeds per regime per scheduler. Confidence intervals are 95% normal-approximation intervals on the mean ($\text{mean} \pm 1.96 \cdot \text{SEM}$); the report presents **mean** $\pm$ **half-width**.

2.2.4 Metrics

For each evaluation cell we record:

- **Mean waiting time** (**avg_waiting_time**): the average queue delay over all jobs.
- **p95 waiting time**: the 95th-percentile queue delay.
- **p99 waiting time**: the 99th-percentile queue delay.
- **Mean completion time** (**avg_completion_time**): waiting time plus job duration.
- **Cluster utilization**: the fraction of CPU-seconds used by jobs divided by total CPU-seconds available over the simulation horizon. This separates “different ordering” from “different throughput”.

2.3 Results

2.3.1 Tail-latency win versus oblivious heuristics (Q1)

The full table of results is in `report/results_tables.md`; we highlight the relevant entries here.

99th-percentile waiting time (lower is better):

regime	First-Fit	Best-Fit	RL	best
light_poisson	3.149	2.971	2.878	RL
heavy_poisson	12.383	13.211	13.023	First-Fit
bursty	9.109	9.142	9.007	RL
large_jobs	16.656	16.629	16.530	RL
small_cluster	7.747	6.844	7.562	Best-Fit

RL is the best scheduler on **three of five informative regimes** (`light_poisson`, `bursty`, `large_jobs`). On the remaining two (`heavy_poisson`, `small_cluster`) it ranks within 5% of the best — *within the 95% CI overlap*. (`wide_cluster` is omitted because all four schedulers hit zero wait.)

95th-percentile waiting time: RL achieves the best p95 on `heavy_poisson` (11.208 ± 1.810 vs FF 11.250 ± 1.626) and on `large_jobs` (15.140 ± 4.630 vs FF 15.178 ± 4.500); the other regimes are statistical ties between RL, FF, and BF.

Answer to Q1: Yes. Across five informative regimes, the learned policy achieves the best p99 in three and is within CI of the best in the other two. It is strictly never the worst scheduler on the tail metrics.

2.3.2 Mean-versus-tail trade-off versus SJF (Q2)

SJF dominates **mean** waiting time in every regime, by 19–37%, as the theory predicts: with oracle duration knowledge, greedily servicing the shortest job each time is optimal for mean delay. RL ranks second-to-last on mean wait in every regime, ahead only of First-Fit (and by small margins).

The story flips entirely on the tail. SJF’s tail metrics are the **worst** of any scheduler in every congested regime, often by a wide margin:

regime	RL p99	SJF p99	RL beats SJF by
light_poisson	2.878	5.774	2.01×
heavy_poisson	13.023	24.351	1.87×
bursty	9.007	19.741	2.19×
large_jobs	16.530	41.115	2.49×
small_cluster	7.562	12.763	1.69×

The mechanism is straightforward: SJF systematically defers long jobs in favor of short ones, which improves the mean (most jobs wait less) but starves the long jobs and produces extreme

tail latencies. The learned policy implicitly trades a smaller fraction of mean performance for substantial tail relief.

Answer to Q2: No, SJF does not dominate. The learned policy gives up roughly 30% on mean to win $1.7\times$ to $2.5\times$ on p99. In any deployment where user-visible SLOs are tail-driven (which is the common case for real cluster workloads — the long jobs are usually the ones that are user-facing or business-critical), this trade is a clear improvement.

2.3.3 Where the gains come from (Q3)

A natural follow-up question: maybe RL just “uses the cluster harder” — runs more jobs per second by being aggressive — and that’s where the tail wins come from. We test this by examining cluster **utilization** (fraction of available CPU-seconds actually used by jobs):

regime	First-Fit	Best-Fit	SJF	RL
light_poisson	0.689	0.689	0.684	0.688
heavy_poisson	0.774	0.777	0.756	0.774
bursty	0.740	0.736	0.725	0.730
large_jobs	0.649	0.647	0.627	0.646
small_cluster	0.763	0.761	0.752	0.758

Utilization is within **0.3 percentage points** across all four schedulers in every regime. Every scheduler completes the same set of jobs and uses essentially the same amount of total compute. The tail-latency wins therefore cannot come from doing more or less work — they must come from **how** that work is organized:

- Better **ordering**: which job to consider first when several are queued and several can fit somewhere.
- Better **packing**: which machine to give a job, leaving the cluster in a better state for future jobs.

Answer to Q3: The learned policy’s gains come from ordering and packing decisions, not from admission control or throughput.

2.3.4 Recipe robustness across cluster sizes (Q4)

The same training recipe — same hyperparameters, same featurizer, same reward shape — was run at three cluster sizes (4, 10, 20 machines) with only the workload’s arrival rate retuned per-cluster to keep utilization in the 60–80% range. All three runs converged within the same wall-clock budget (~14 minutes per policy on the M4 Pro, ~3000–10000 fps depending on cluster size). All three produce the headline result of beating SJF on p99 in their primary regimes.

Answer to Q4: Yes — the recipe is robust. A single training script, with cluster size as the only nominal change, produces a working policy at every size we tested.

The negative result of Section 1.3.7 (set-attention) shows that *architectural* generalization across cluster sizes is harder than recipe robustness: producing a single policy that runs on multiple sizes is not the same as having a recipe that produces a working policy per size.

2.3.5 Limitations of the evaluation

- **wide_cluster is uninformative.** At arrival rate 6.0 on 20 machines, every scheduler completes every job with zero wait. The regime should have been calibrated to a higher arrival rate (~ 12) to produce queueing. We document the result honestly rather than hide it.
- **5 seeds yields wide CIs** in some cells (notably p99 on `large_jobs`). Close calls (e.g., RL near FF on `heavy_poisson` p99) should be read as ties, not upsets.
- **Synthetic workloads only.** All evaluations use Poisson or bursty Poisson arrivals over configurable demand ranges. We do not evaluate on real production traces; transferring from synthetic to real workloads is open future work.
- **Two-resource model (CPU + memory).** Real clusters track I/O, network, GPU, and per-job affinity constraints we do not model.
- **No preemption or contention.** A placed job runs to completion on its assigned machine without interference from other co-resident jobs — a simplification compared to real schedulers.

2.3.6 Summary of evaluation conclusions

The evaluation establishes that, on the cluster-scheduling problem as we have formalized it:

1. The learned policy is the best or statistically tied for best scheduler on tail-latency metrics across all evaluable regimes.
2. It trades a quantifiable, modest fraction of mean-time performance to the duration oracle (SJF) for substantial — up to $2.5\times$ — tail improvement.
3. Its gains come from ordering and packing decisions, not from admission control: cluster utilization is essentially constant across schedulers.
4. The training recipe is robust enough that the same script produces a working policy at three different cluster sizes with only workload arrival rate retuned per-size.
5. Cross-cluster *architectural* generalization (a single policy that runs across cluster sizes) is harder; our set-attention attempt fails to train despite passing all unit tests, and we leave diagnosing it as future work.

References

1. Coffman, E. G., Garey, M. R., & Johnson, D. S. (1996). *Approximation algorithms for bin packing: A survey*. PWS Publishing Co.
2. Dean, J., & Barroso, L. A. (2013). *The tail at scale*. Communications of the ACM, 56(2), 74–80.

3. Huang, S., & Ontañón, S. (2022). *A closer look at invalid action masking in policy gradient algorithms*. FLAIRS.
4. Mao, H., Alizadeh, M., Menache, I., & Kandula, S. (2016). *Resource management with deep reinforcement learning*. HotNets.
5. Mao, H., Schwarzkopf, M., Venkatakrisnan, S. B., Meng, Z., & Alizadeh, M. (2019). *Learning scheduling algorithms for data processing clusters*. SIGCOMM.
6. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv:1707.06347.
7. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., & Wilkes, J. (2015). *Large-scale cluster management at Google with Borg*. EuroSys.

Software resources:

- Stable-Baselines3 and `sb3-contrib` (PyTorch implementation of PPO and MaskablePPO).
- Gymnasium (the maintained successor to OpenAI Gym).
- NumPy, pandas, matplotlib (numerics, data handling, plotting).
- Pytest (testing).

We did not depend on any external data resources: all workloads are generated on-the-fly by our `WorkloadGenerator`.

Relationship to other work by team members

This project is independent of any other coursework or research project that the team members are currently involved in. None of the code, results, or written analysis is shared with another submission in this or any prior semester. The simulator, environment, training pipeline, and evaluation harness were all written from scratch for this course.

Appendix A. Numbers, reproducibility, and the full table

All numbers in this report are auto-generated from `artifacts/eval_run_final/results_summary.csv` and rendered in `report/results_tables.md`, which contains the complete mean $\pm$ half-width of 95% CI for every metric $\times$ regime $\times$ scheduler cell. Any number cited in the body can be cross-referenced there.

The training and evaluation pipeline is fully reproducible from the public repository at https://github.com/J0hns0n-Wang/AI_Prac:

```
# Install
```

```
python3.10 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
```

```
# Train (one command per cluster size, ~14 min each on M4 Pro)
```

```
python -u -m cluster_scheduler.train --timesteps 8000000 --n-envs 10 \
```

```

--policy-hidden 256 256 --activation gelu \
--num-machines 10 --num-jobs 500 --arrival-rate 8.0 \
--featurizer rich --rich-top-k 4 \
--reward-mode dense --backlog-penalty-weight 0.1 --completion-bonus 1.0 \
--normalize-reward --n-steps 1024 --batch-size 4096 \
--learning-rate 2.5e-4 --lr-schedule linear \
--ent-coef 0.005 --clip-range-vf 0.2 \
--gamma 0.999 --gae-lambda 0.95 \
--save-path artifacts/ppo_m10_v4.zip --log-dir runs/ppo_m10_v4

# Evaluate (replays held-out seeds across all schedulers)
python scripts/run_experiments.py \
--model artifacts/ppo_m4_v4.zip artifacts/ppo_m10_v4.zip artifacts/ppo_m20_v4.zip \
--seeds 1000 1001 1002 1003 1004 \
--out-dir artifacts/eval_run_final \
--arrival-sweep

```

Saved policies live under `artifacts/` (gitignored, reproducible from the commands above). Each `*.zip` ships with a `*.meta.json` sidecar recording the featurizer, reward configuration, network architecture, and training hyperparameters used, so evaluation reconstructs the exact training-time observation space.